AI agents create change faster

than teams can verify it.

The bottleneck moved from typing code to proving that a change is safe, complete, and worth human attention.

SECONDS	**to generate a large patch**
HOURS	**to understand and verify it**
ONE GAP	**objective evidence before review**

Swipe to see how AIV turns an untrusted change into a verifiable decision.

Why existing solutions do not close the gap

Compiler / SAST / secret scanner

Excellent specialists, but they run as separate jobs and reports. They do not create one protected evidence receipt for the change.

LLM reviewer

Useful for contextual judgement, but its answer can vary with prompt, model, context, and time. That is not enough for the only hard gate.

SDLC orchestrator / policy engine

They sequence work or decide allow/deny. They still need trustworthy code facts as input.

The gap **One diff-scoped, policy-bound evidence receipt that composes these tools.**

THE SOLUTION

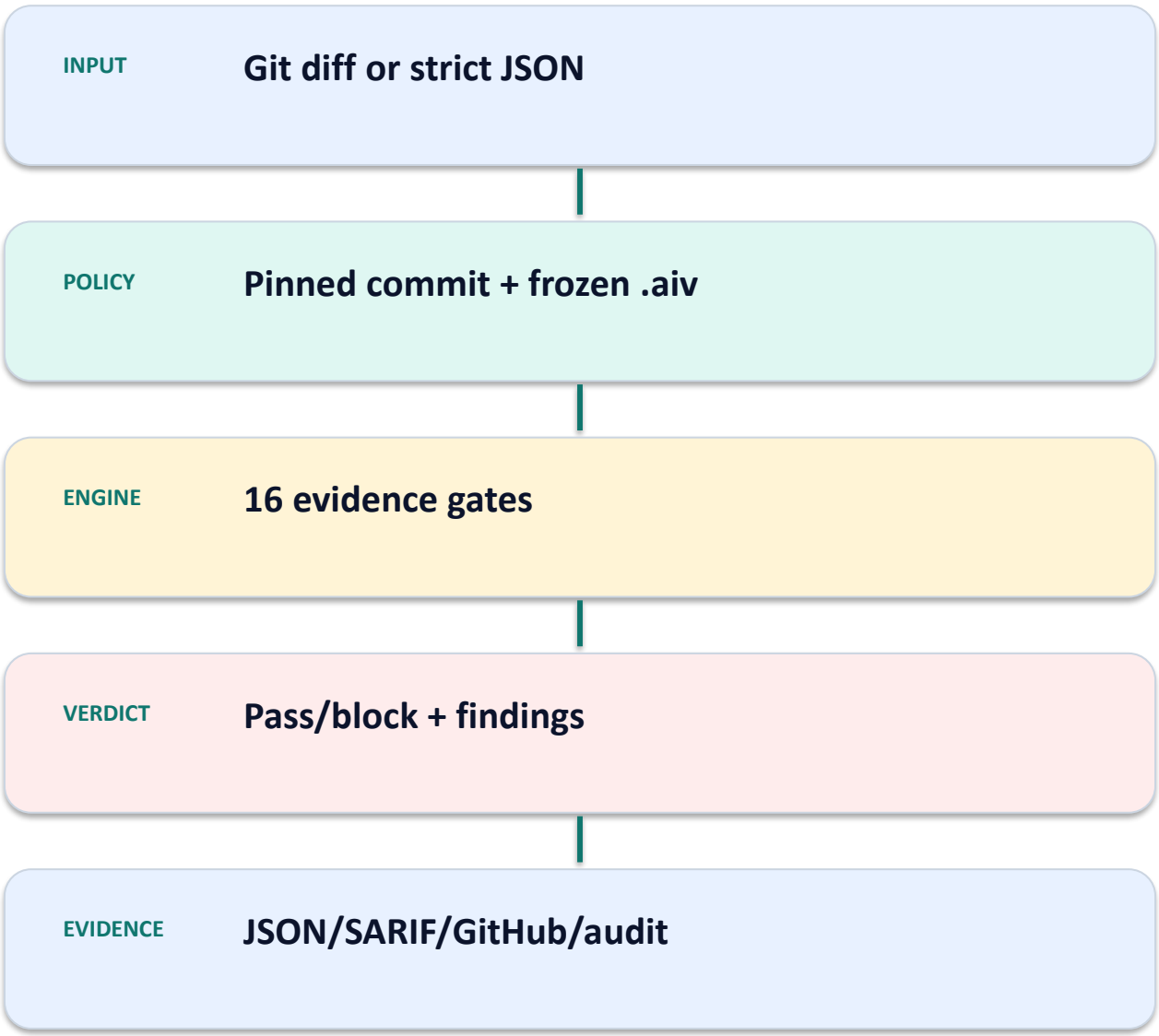
AIV turns change into verifiable evidence

1. Capture the proposed diff
2. Pin one trusted policy commit
3. Freeze every .aiv policy file
4. Run objective and advisory gates
5. Publish one structured verdict

AIV does not identify who wrote the code. It asks whether the change is backed by evidence.

ARCHITECTURE

How AIV works end to end



The source workspace and policy workspace stay separate throughout evaluation.

16 gates. Four evidence jobs.

CHANGE INTEGRITY

syntax • invariant
dependency • package existence

BEHAVIORAL EVIDENCE

test adequacy • security
design • generic rules

REVIEW FOCUS

density • cohesion
duplication • external SARIF

GOVERNANCE

documentation • PR metadata
license header • advisory Jev

Hard facts can block. Structural or judgement-heavy signals remain advisory.

GATE FAMILY 1

Stop guaranteed breakage first

syntax

Strict parse result or visible unverified parser/toolchain state.

invariant

Conflict markers, placeholders, agent edit artifacts, assistant chatter, and tautology tests.

dependency

Java and Python imports must be backed by manifest, standard-library, or first-party evidence.

package-existence

New package declarations are classified as found, absent, or unverified against a registry/mirror.

Use: local + CI. Registry access is an explicit external input.

GATE FAMILY 2

Demand evidence around behavior

test-adequacy

Added test functions must contain assertion, expectation, or verification evidence.

security

High-confidence credential formats and keyword-bound secret candidates on added lines.

design

Repository-owned forbidden/required architecture patterns from frozen YAML policy.

rules

Project or framework pattern, call-argument, and hollow-function checks in generic YAML.

Use: CI hard gates after repository calibration. Missing-test evidence remains advisory.

GATE FAMILY 3

Focus scarce reviewer attention

density

Highlights unusually thin Java logic or low-entropy structure.

cohesion

Highlights changes spread across many repository areas or files.

duplication

Finds exact normalized added blocks already present in existing source.

external

Runs trusted tools and folds their SARIF findings into the same severity-aware receipt.

Use: advisory review routing. External tools may be hard only when the project already trusts them.

GATE FAMILY 4

Make delivery policy visible

doc-integrity

Checks local links, paths, cross-references, required mentions, and command completeness.

pr-metadata

Checks title, body, untouched template text, and optional disclosure structure.

license-header

Checks approved project markers on newly added source files.

jev

Optional model-backed yes/no questions. Always advisory and declared as an external input.

Use: enable only the governance surfaces your repository owns and can maintain.

TRUST BOUNDARY

The change cannot edit its own yardstick

UNTRUSTED

Proposed source, manifests, tests, docs, and metadata

TRUSTED

One base ref resolved once to an immutable commit OID

Freeze the complete policy tree

AIV materializes regular .aiv Git blobs into a private snapshot. It rejects drift, ignored extras, symlinks, type changes, and path escapes.

Bind the decision

Request bytes + canonical config + exact frozen sidecars + policy commit produce the evidence digest.

The bytes consumed by the gates are the bytes named by the evidence.

INTEGRATION

One verdict. Multiple delivery surfaces.

Developer

terminal + exit code

CI

JSON + SARIF

GitHub

Checks + comments + labels

Agent / MCP host

aiv verify schema v2

AI-DLC

advisory or fail-closed sensor

Governance

audit chain + HTML/JSONL report

No publisher reruns a gate. Every surface consumes the same AIVResult.

AIV fits inside an AI-driven lifecycle

1. **Agent produces code or a full diff request**
2. **AI-DLC invokes the AIV sensor**
3. **AIV pins policy and computes evidence**
4. **AI-DLC records, advances, or routes intervention**

Two contracts

Advisory mode reports without blocking. Blocking mode requires trusted policy, fails closed, and rejects undeclared external inputs.

USE CASES

Where AIV creates value

Before commit

Catch objective edit/parse/secret/test failures before a pull request exists.

Pull-request CI

Require trusted policy and publish evidence where reviewers already work.

Agent and platform workflows

Give coding agents, MCP hosts, AI-DLC, and policy engines one stable verification contract.

Audit and calibration

Retain verdict history, anchor the chain head, and tune rules with reviewer labels.

The common outcome: less human time spent proving basic facts.

PRODUCT EVIDENCE

What is verified today

25 Maven reactor projects

844 tests

0 failures / errors

0 critical / high review findings

- Full clean Maven verification and configured coverage gates pass.
- AI-DLC adapter tests, preset parity, YAML, docs, visuals, and deck validators pass.
- Independent adversarial review ended at Critical 0 / High 0.
- One Windows capability-gated symlink test is skipped where the OS cannot create a link.

FOUR STEPS

Adopt without boiling the ocean

1. Observe

Start with minimal local syntax + invariant.

2. Calibrate

Add repository gates; keep structural signals advisory.

3. Protect

Commit .aiv; require pinned/frozen trusted policy in CI.

4. Enforce

Promote only objective, proven rules. Retain evidence.

Enable the gates that solve your problem. Do not enable everything because it exists.

Pilot one repository.

Measure one thing:

How much reviewer time stops being spent proving basic facts?

- Choose a real change workflow.
- Run advisory first.
- Protect policy in CI.
- Review the evidence with maintainers.
- Let measured value decide the next step.

github.com/vaquarkhan/aiv-integrity-gate

Viquar Khan | vaquar.khan@gmail.com